

Wireshark Network Analysis

Exercise 6.1 — TCP Three-Way Handshake (Layer 4 — Transport)

- **Observation:** Using filter `tcp.flags.syn == 1` or `(tcp.flags.ack == 1 and tcp.flags.syn == 0 and tcp.seq == 0 and tcp.ack == 1)`, a handshake was captured between curl and the FastAPI backend on port 8000.
 - **Packet 255 (SYN):** Client initiates connection; Seq=0, Win=65535, Len=0.
 - **Packet 256 (SYN, ACK):** Server acknowledges client and sends its own SYN; Seq=0, Ack=1, Win=65535.
 - **Packet 258 (ACK):** Client acknowledges server's SYN; Seq=1, Ack=1, Win=65280, establishing connection.
- **OSI Layers:** L4 (Transport) manages sequence number synchronization. L3 (Network) handles logical routing between loopback source/destination 127.0.0.1. L2 (Data Link) uses NULL/Loopback encapsulation for local traffic.
- **Distributed Concept:** Guarantees reliable connection establishment. In Campus Ride, before the Next.js frontend or database exchange core application data, TCP ensures nodes confirm reachability and track sequence delivery, preventing data loss to unavailable services.

Exercise 6.2 — HTTP GET with Full Layer Expansion (Layers 1–7)

- **Observation:** Capturing a GET `/api/v1/students/` request via curl revealed the complete, plaintext protocol stack across all OSI layers.
 - **Frame/Physical (L1):** 138 bytes (1,104 bits) captured on wire.
 - **Data Link (L2):** NULL/Loopback encapsulation with no standard Ethernet MAC addresses present.
 - **Network (L3):** IPv4 header showing source and destination as 127.0.0.1.
 - **Transport (L4):** TCP header showing Source Port 52734 to Destination Port 8000 using PSH+ACK flags to push data instantly.
 - **Presentation (L6) & Application (L7):** ASCII-encoded HTTP text containing the stateless REST API call to retrieve student records.
- **Distributed Systems Concept:** Demonstrates client-server communication using a stateless REST API. Because HTTP is stateless and requests are independent, this architecture allows individual containerized backend pods to handle requests flexibly without needing prior session context, enabling smooth horizontal scaling in Kubernetes.

Exercise 6.3 — HTTP POST with JSON Payload (Layer 6 — Presentation)

- **Observation:** Captured student registration POST (Packet 4204, 304 bytes, TCP Segment Len 260) targeting `/api/v1/students/`.
 - **HTTP Headers:** Content-Type: application/json, Content-Length: 112.
 - **JSON Payload:** Text data strings containing student attributes (name: Norazmirah binti Sulimin, matric_id: 304008, email, lat, lng).
- **OSI Layers:** L4 (Transport) sets PSH flag to bypass system buffers for immediate delivery. L6 (Presentation) handles JSON serialization, converting structured objects into a standard transmittable string format. L7 (Application) carries the REST resource creation payload.
- **Distributed Concept:** Illustrates inter-service interoperability. JSON acts as the language agnostic data contract between a TypeScript frontend (Next.js) and a Python backend (FastAPI). Relying on a shared protocol contract ensures heterogeneous, isolated container components decode parameters accurately.

Exercise 6.4 — TCP Four-Way Termination (Layer 4 — Transport)

- **Observation:** Connection teardown analyzed using filter `tcp.stream eq 13` captured a simultaneous close pattern.
 - **Packet 2362 & 2363:** Client and server send [FIN, ACK] concurrently, signaling completion of data transmission.
 - **Packets 2364–2366:** Exchange of final acknowledgments (ACK), completing connection termination.
- **OSI Layers:** L4 (Transport) manages termination via FIN/ACK flags to ensure all remaining data in flight is acknowledged. L3 (Network) continues routing the final termination packets between the endpoints.
- **Distributed Concept:** Graceful resource release. For systems with high concurrent traffic, cleanly terminating connections prevents unreleased "half-open" connection channels from accumulating on the server. This protects file descriptors and RAM from exhaustion, maintaining service availability.

Exercise 6.5 — Docker Container-to-Container Traffic (L2/L3 Networking)

- **Observation:** Docker network inspection of bridge network `dist_sys_app_default` exposed a private 172.18.0.0/16 subnet with Gateway 172.18.0.1.
 - **IP Assignments:** Postgres (172.18.0.2), Backend (172.18.0.3), Frontend (172.18.0.4). Inter-service calls run entirely inside the isolated virtual bridge switch without external network exposure.
- **OSI Layers:** L2 (Data Link) utilizes virtual MAC addresses (e.g., Postgres MAC: e6:2e:e4:dd:cb:fb) for local bridge frame switching. L3 (Network) routes traffic through private container IPs, leveraging Docker's embedded DNS engine (127.0.0.11) to translate string service names to IPs.
- **Distributed Concept:** Service discovery and network isolation. Container IPs are dynamic and change on every reboot. DNS-based discovery ensures the backend dynamically resolves the database destination target by its service name (postgres) rather than a fragile hardcoded IP. This matches the structural architecture managed by kube-dns in Kubernetes clusters.

Exercise 6.6 — TLS/HTTPS Analysis (Railway Production)

- **Observation:** Traffic captured on a live cloud environment (<https://frontend-production-1637.up.railway.app>) contrasted local plaintext behavior.
 - **TLS Handshake:** Begins with *Client Hello* (supported cipher suites), followed by *Server Hello* delivering an automated Let's Encrypt TLS security certificate.
 - **Payload Encryption:** All subsequent frames are designated as {Encrypted Application Data}. The HTTP paths, headers, payload variables, and JSON strings are completely secure and unreadable to sniffers.
- **OSI Layers:** L4 ensures reliable packet transport. L5 (Session) manages security session negotiation lifecycles. L6 (Presentation) transforms plaintext HTTP records into ciphertext. L7 (Application) contains the encrypted application request layer.
- **Distributed Concept:** Transport security for identity verification. Local unencrypted HTTP passes authentication structures like Supabase JWT tokens in plaintext, risking extraction by an attacker. Cloud TLS encryption blocks interception, working alongside Supabase Row-Level Security (RLS) data isolation rules to build a secure end-to-end distributed authorization framework.